TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐHQG-HCM

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC

CSC13002 – LẬP TRÌNH ỨNG DỤNG JAVA

ĐỒ ÁN #1

SLANG DICTIONARY

Ứng dụng từ điển slang word bằng Java Swing

Giảng viên phụ trách	Nguyễn Văn Khiết (LT) – Nguyễn Đức Huy (TG) – Hồ Tuấn Thanh (TH)
Sinh viên thực hiện	.....
MSSV	.....
Lớp	.....
Thời gian	Tháng 11 năm 2026

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

1. GIỚI THIỆU....3
2. PHÂN TÍCH YÊU CẦU....4
3. KIẾN TRÚC VÀ THIẾT KẾ....5
4. CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT....7
5. GIAO DIỆN VÀ CHỨC NĂNG....9
6. KIỂM THỬ VÀ ĐÁNH GIÁ....15
7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....17
8. TÀI LIỆU THAM KHẢO....17
PHỤ LỤC A – TỰ ĐÁNH GIÁ....17
PHỤ LỤC B – CHẠY VÀ ĐÓNG GÓI....18
PHỤ LỤC C – VIDEO VÀ COMMIT....18

1. GIỚI THIỆU

1.1. Bối cảnh và mục tiêu

Đồ án #1 của môn CSC13002 yêu cầu xây dựng một ứng dụng từ điển slang word bằng Java, vận dụng Java IO, lập trình hướng đối tượng, Generic và Collections. Ứng dụng phải hỗ trợ tra cứu nhanh, lưu dữ liệu sau khi thay đổi và đáp ứng 10 chức năng theo mô tả của đề bài.

Mục tiêu của đồ án là tạo ra một ứng dụng desktop hoàn chỉnh, có giao diện dễ sử dụng, dữ liệu được tổ chức hợp lý và có khả năng mở rộng. Báo cáo này trình bày quá trình phân tích, thiết kế, cài đặt, kiểm thử và đánh giá kết quả.

1.2. Phạm vi

- Dữ liệu đầu vào: file slang_clean.txt (đã chuẩn hoá từ slang.txt), gồm 7.638 mục từ và 8.941 định nghĩa.
- Ứng dụng desktop Java Swing, chạy offline, không cần máy chủ hay kết nối mạng.
- Hỗ trợ 10 chức năng: tra cứu theo word/definition, lịch sử, thêm, sửa, xóa, reset, random và hai chế độ đố vui.
- Kiểm thử hiệu năng với dữ liệu sinh tự động 100.000 slang word.
- Giao diện chỉ dùng đen–trắng (thang xám), không sử dụng màu nhấn.

1.3. Công nghệ sử dụng

Thành phần	Công nghệ / lựa chọn
Ngôn ngữ	Java 17+ (biên dịch với --release 17, kiểm thử trên JDK 23)
Giao diện	Java Swing + Java2D (component bo góc, vẽ trạng thái bằng viền/nét đứt)
Lưu trữ	Java IO: text UTF-8 và snapshot nhị phân DataInputStream/DataOutputStream
Cấu trúc dữ liệu	HashMap, LinkedHashMap, ArrayList, ArrayDeque, posting list trigram
Kiểm thử	Bộ test tự viết, không phụ thuộc thư viện ngoài; đo thời gian bằng nanoTime
Đóng gói	jar (JDK), PowerShell build script, Git

Ứng dụng không sử dụng thư viện bên ngoài. Toàn bộ giao diện, chỉ mục và định dạng lưu trữ đều được cài đặt trong mã nguồn của đồ án.

2. PHÂN TÍCH YÊU CẦU

2.1. Yêu cầu chức năng

STT	Chức năng	Mô tả ngắn
1	Tìm theo slang word	Tra cứu chính xác, không phân biệt hoa thường; hiển thị đầy đủ định nghĩa.
2	Tìm theo definition	Hiển thị tất cả slang word có định nghĩa chứa chuỗi con; highlight từ khoá.
3	Lịch sử tra cứu	Lưu từ khoá, loại tra cứu, số kết quả, thời gian; xóa được.
4	Thêm slang word	Kiểm tra trùng; cho chọn ghi đè, thêm bản trùng hoặc hủy.
5	Chỉnh sửa	Chọn đúng mục từ, sửa word/định nghĩa, lưu ngay.
6	Xóa	Chọn mục từ, xác nhận trước khi xóa.
7	Reset	Khôi phục danh sách từ slang_clean.txt (bản chuẩn hoá), có xác nhận.
8	Random	Hiển thị một slang word ngẫu nhiên (on this day).
9	Đố vui word → nghĩa	Một slang word và 4 định nghĩa; chọn đáp án, phản hồi đúng/sai.
10	Đố vui nghĩa → word	Một định nghĩa và 4 slang word; chọn đáp án, phản hồi đúng/sai.

2.2. Yêu cầu phi chức năng

Nhóm	Yêu cầu	Cách đáp ứng
Hiệu năng	Tra cứu dưới 1 giây với 100.000 slang word	HashMap O(1) cho word; trigram inverted index cho definition; benchmark 100k.
Lưu trữ	Không đánh chỉ mục lại ở lần chạy sau	Snapshot nhị phân chứa cả word index và posting list; kiểm tra fingerprint trước khi dùng.
Toàn vẹn	Không mất dữ liệu khi add/edit/delete	Ghi file tạm rồi move atomic; lưu cả text và snapshot; có fallback rebuild khi snapshot hỏng.
Giao diện	Menu rõ ràng, dễ dùng, chỉ đen–trắng	Menu bar + sidebar + phím tắt; Theme grayscale, trạng thái đúng/sai bằng viền đậm/nét đứt.
Nền tảng	Chạy độc lập, không cần mạng	Jar tự chứa dữ liệu mẫu; dữ liệu làm

Nhóm	Yêu cầu	Cách đáp ứng
		việc lưu trong thư mục data.

2.3. Các luồng nghiệp vụ chính

Tra cứu

1. Chuẩn hoá từ khoá (NFC, gộp khoảng trắng, chữ thường).
2. Nếu tìm theo word: tra HashMap<String, List<Slang>> để lấy tất cả bản ghi trùng tên.
3. Nếu tìm theo definition: lấy giao posting list trigram, kiểm tra lại từng định nghĩa.
4. Hiển thị kết quả, thời gian chạy và ghi vào lịch sử.

Thêm / sửa / xóa

1. Kiểm tra dữ liệu nhập và chuẩn hoá.
2. Nếu từ đã tồn tại khi thêm: hỏi người dùng chọn Ghi đè, Thêm bản trùng hoặc Hủy.
3. Cập nhật SlangDictionary, ghi data/slang_modified.txt và snapshot.
4. Thông báo kết quả qua thanh trạng thái; làm mới các màn hình liên quan.

Đố vui

1. Chọn ngẫu nhiên một mục từ và một định nghĩa làm đáp án đúng.
2. Sinh 3 đáp án nhiễu duy nhất, không trùng đáp án đúng.
3. Trộn 4 đáp án; khi người dùng chọn, đánh dấu đáp án đúng bằng viền đậm, đáp án chọn sai bằng viền nét đứt, các đáp án khác chuyển xám.

3. KIẾN TRÚC VÀ THIẾT KẾ

3.1. Sơ đồ tổng quan

Ứng dụng được chia thành các tầng độc lập: model, data, service và ui. Cửa sổ chính đóng vai trò điều phối, các panel chỉ giao tiếp qua interface AppHost.

```
App / ScreenshotMode
       |
       v
  MainFrame (menu, sidebar, CardLayout)
       |  AppHost
       v
  Home / Search / Add / Edit / Delete / Reset / Random / Quiz panels
       |
       v
  SlangDictionary  <---->  DictionaryStore  ---->  data/*.bin, *.txt
       ^
       |
  HistoryService, QuizService
```

3.2. Thành phần chính

Lớp / package	Trách nhiệm
model.Slang	Một mục từ: id, word, danh sách định nghĩa, thời gian; implements Comparable.
model.HistoryEntry	Một lượt tra cứu: thời gian, loại, từ khoá, số kết quả.
model.QuizQuestion	Câu hỏi 4 đáp án, chỉ số đáp án đúng, giải thích.
data.SlangDictionary	Quản lý kho mục từ, word index, trigram index, thao tác thêm/sửa/xóa.
data.TrigramIndex	Xây dựng và truy vấn posting list trigram cho definition.
data.DictionaryStore	Đọc/ghi slang.txt/slang_clean.txt, slang_modified.txt, dictionary.bin; fingerprint và fallback.
service.HistoryService	Quản lý lịch sử 200 mục, lưu/đọc data/history.txt.
service.QuizService	Sinh câu hỏi 4 đáp án duy nhất cho hai chế độ đố vui.
ui.MainFrame	Cửa sổ chính, menu, sidebar, điều hướng CardLayout, thanh trạng thái.
ui.Theme / components	Bảng màu grayscale, font, nút/panel/ô nhập/option quiz tự vẽ.

3.3. Luồng khởi động

1. App khởi tạo Look & Feel, font và các thiết lập grayscale.
2. Bootstrap xác định thư mục dự án; nếu thiếu dữ liệu thì trích xuất slang.txt / slang_clean.txt đi kèm jar.
3. DictionaryStore kiểm tra data/slang_modified.txt và data/dictionary.bin.
4. Nếu fingerprint khớp: đọc snapshot, không đánh chỉ mục lại. Nếu không: parse text, xây chỉ mục, ghi snapshot mới.
5. MainFrame được tạo trên EDT; HomePanel hiển thị thống kê và nạp dữ liệu.

3.4. Thiết kế giao diện

Giao diện dùng một cửa sổ chính với menu bar ở trên, sidebar điều hướng bên trái và vùng nội dung thay đổi bằng CardLayout. Mỗi chức năng là một panel riêng, kế thừa Refreshable để làm mới khi được mở hoặc khi dữ liệu thay đổi.

Theo yêu cầu, toàn bộ giao diện chỉ dùng đen–trắng: các màu trong Theme có R = G = B. Trạng thái không dùng màu để phân biệt; thay vào đó dùng độ đậm của viền, nền xám, nét đứt và nội dung chữ. Ảnh trong báo cáo cũng được chuyển sang thang xám.

4. CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

4.1. Bài toán

Với 100.000 slang word, nếu chỉ dùng List thì mỗi lần tìm kiếm phải duyệt tuần tự, không đáp ứng yêu cầu dưới 1 giây và còn bị giới hạn điểm theo quy định của đề. Đồ án sử dụng thêm HashMap, LinkedHashMap, ArrayDeque và chỉ mục nghịch đảo trigram.

Dữ liệu đầu vào được kiểm tra và chuẩn hoá trước khi đưa vào ứng dụng. File gốc slang.txt (7.656 mục từ, 8.959 định nghĩa) được giữ nguyên để đối chiếu; file slang_clean.txt (7.638 mục từ, 8.941 định nghĩa) là dữ liệu chuẩn mà ứng dụng sử dụng. Quá trình chuẩn hoá đã sửa header, gộp 5 dòng thiếu key, xoá 15 bản ghi trùng hoàn toàn, gộp 3 cặp key có nghĩa khác, trim 14 key và 14 định nghĩa, chuẩn hoá 11 dòng dấu |, ánh xạ 2 key non-ASCII, sửa 4 định nghĩa bị cắt cụt và đổi chỗ 2 cặp sai thứ tự A–Z. Chi tiết theo từng dòng nằm trong report/SLANG_ERRORS_REPORT.md và file CSV đi kèm.

4.2. Chỉ mục từ khoá

Mỗi mục từ có một id duy nhất; nhiều mục từ có thể trùng tên. Vì vậy cấu trúc chính là:

```
LinkedHashMap<Long, Slang> byId;              // giữ thứ tự chèn
HashMap<String, List<Slang>> byWord;          // key = word đã chuẩn hoá
List<Slang> ordered;                          // thứ tự dùng cho posting list
```

- Tra cứu chính xác trung bình O(1); danh sách kết quả vẫn giữ được các bản ghi trùng.
- Word được chuẩn hoá bằng Unicode NFC, gộp khoảng trắng và lowercase(Locale.ROOT).
- Gợi ý theo prefix duyệt các key của byWord, sắp xếp theo độ dài rồi alphabet.

4.3. Chỉ mục trigram cho định nghĩa

Mỗi định nghĩa được chuẩn hoá và tách thành các trigram (3 ký tự liên tiếp). Mỗi trigram ánh xạ tới danh sách chỉ số các mục từ có chứa trigram đó.

```
for each entry:
    for each trigram in entry.searchText:
        postings[trigram].add(entryIndex)

query(keyword):
    if keyword.length < 3: scan entries
    else:
        candidates = intersection(postings[t] for t in trigrams(keyword))
        for candidate in candidates:
            if candidate.matchesDefinition(keyword): add candidate
```

Bước kiểm tra lại bằng matchesDefinition loại bỏ false-positive do các trigram xuất hiện ở vị trí khác nhau. Việc kiểm tra được thực hiện trên từng định nghĩa riêng, nên một cụm từ không thể khớp "xuyên" qua ranh giới hai định nghĩa.

4.4. Độ phức tạp

Thao tác	Cấu trúc / chi phí	Ghi chú
Tìm theo word	HashMap, O(1) trung bình	O(k) với k bản ghi trùng tên
Tìm theo definition (≥3 ký tự)	Giao posting list + kiểm tra	Thực tế 1–20 ms với 100k
Tìm theo definition (<3 ký tự)	Quét tuần tự	Vẫn dưới 1 giây với dữ liệu trong bộ nhớ
Thêm / sửa / xóa	Cập nhật map + rebuild chỉ mục	Thao tác người dùng, không phải truy vấn
Lưu / nạp	Text UTF-8 + snapshot nhị phân	Nạp snapshot ~0,6 giây với 100k

4.5. Lưu trữ và phục hồi

Dữ liệu được lưu ở hai dạng:

- slang_modified.txt: text UTF-8, mỗi dòng word`definition1|definition2, có thể mở bằng editor.
- dictionary.bin: snapshot nhị phân chứa entries, word index và toàn bộ posting list trigram.

Snapshot lưu kèm CRC32, kích thước và tên file nguồn. Khi khởi động, nếu fingerprint khớp, ứng dụng đọc thẳng snapshot. Nếu snapshot cũ, hỏng hoặc file text đã thay đổi, chương trình tự parse lại text, xây chỉ mục và ghi snapshot mới. File được ghi qua file tạm rồi move atomic.

4.6. Lịch sử tra cứu

HistoryService dùng ArrayDeque<HistoryEntry>, thêm mục mới vào đầu, giới hạn 200 mục và ghi data/history.txt sau mỗi lần tra cứu. Định dạng text có escape ký tự tab/newline.

4.7. Vì sao đáp ứng yêu cầu điểm

Ứng dụng không chỉ dùng List: HashMap, LinkedHashMap, ArrayDeque, HashSet, posting list int[] và chỉ mục nghịch đảo trigram đều được sử dụng. Đây là căn cứ để đồ án không bị giới hạn ở mức 70 điểm theo quy định "chỉ dùng List".

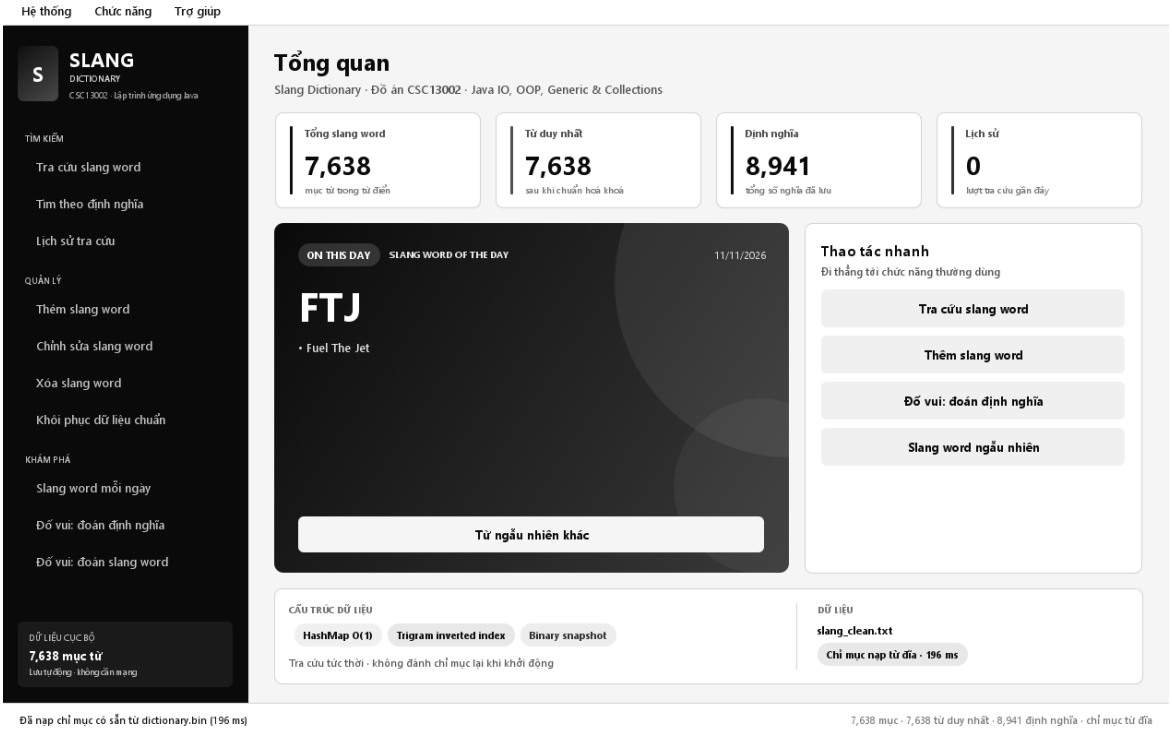
5. GIAO DIỆN VÀ CHỨC NĂNG

5.1. Nguyên tắc đen–trắng

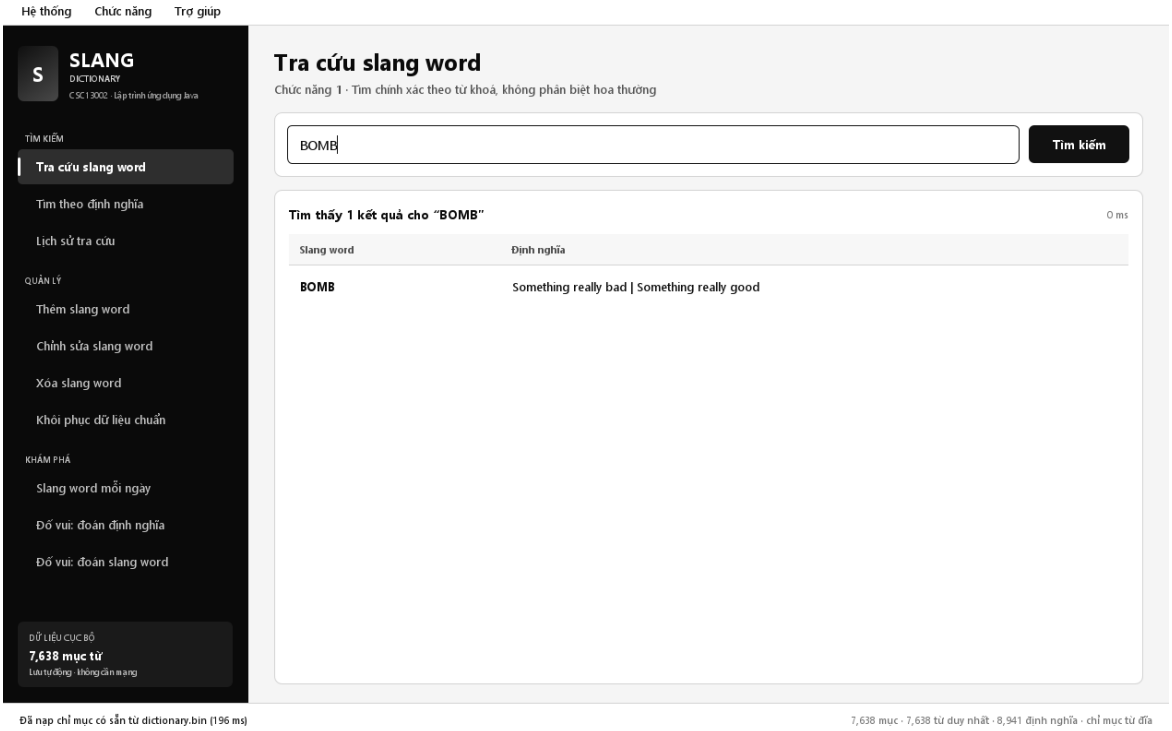
Toàn bộ Theme sử dụng các giá trị xám (R = G = B). Không có màu xanh, đỏ, tím hay vàng. Các trạng thái được phân biệt bằng:

- Đậm/nhạt của nền và viền.
- Viền liền đậm cho đáp án đúng; viền nét đứt cho đáp án chọn sai.
- Chữ mô tả trực tiếp: "Chính xác", "Chưa đúng", "Đáp án đúng: …".
- Badge A/B/C/D chuyển đen hoặc xám tùy trạng thái.

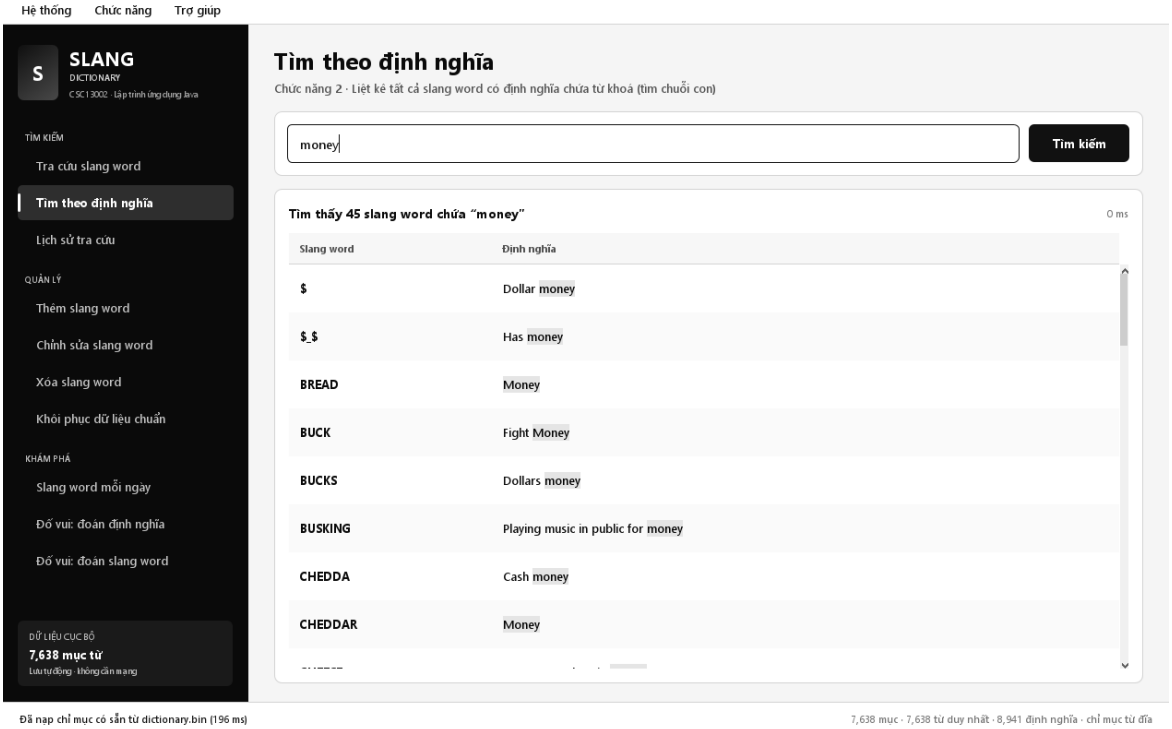
Ảnh chụp trong báo cáo được chuyển sang grayscale bằng Java2D trước khi ghi, nên không còn pixel lệch kênh màu do ClearType.



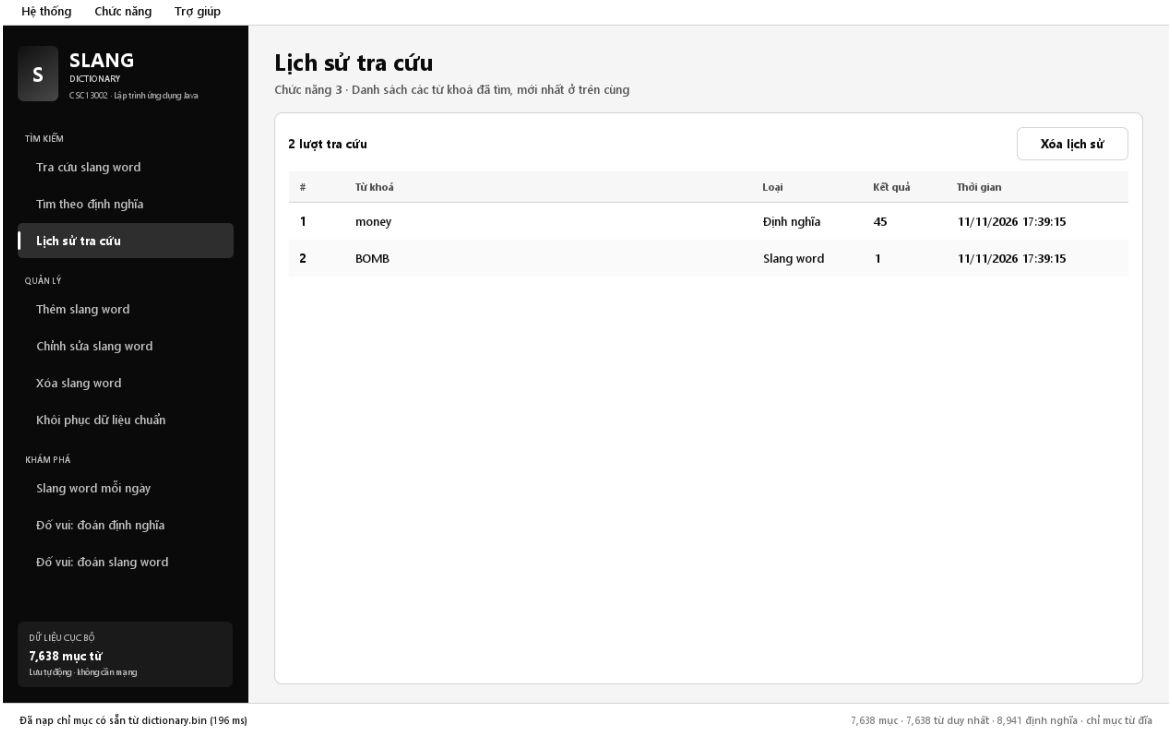
Hình 5.1. Trang tổng quan với thống kê, từ của ngày và thao tác nhanh



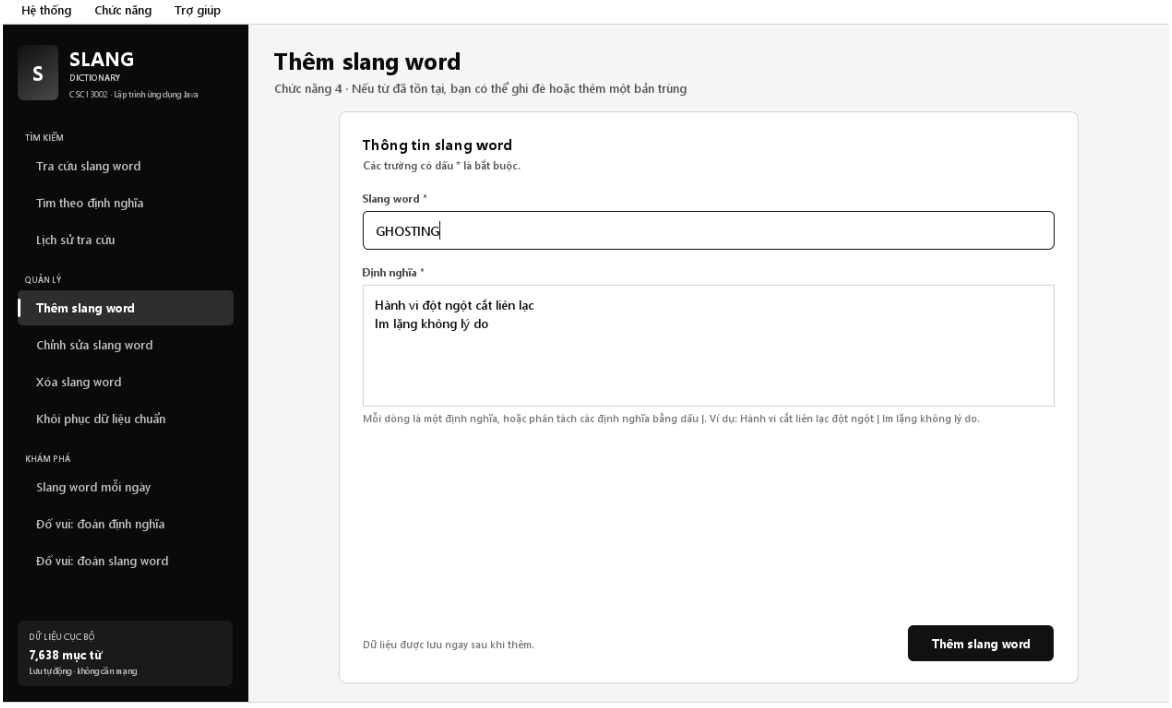
Hình 5.2. Tra cứu chính xác theo slang word



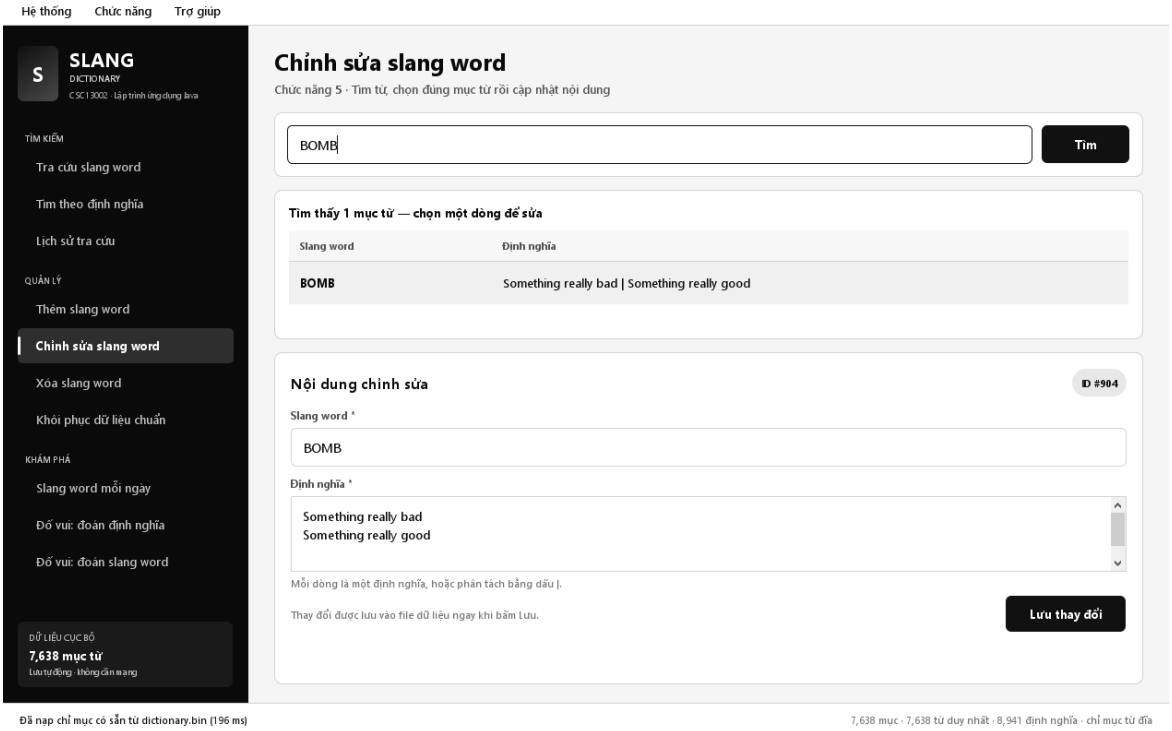
Hình 5.3. Tìm theo định nghĩa, highlight từ khoá bằng nền xám



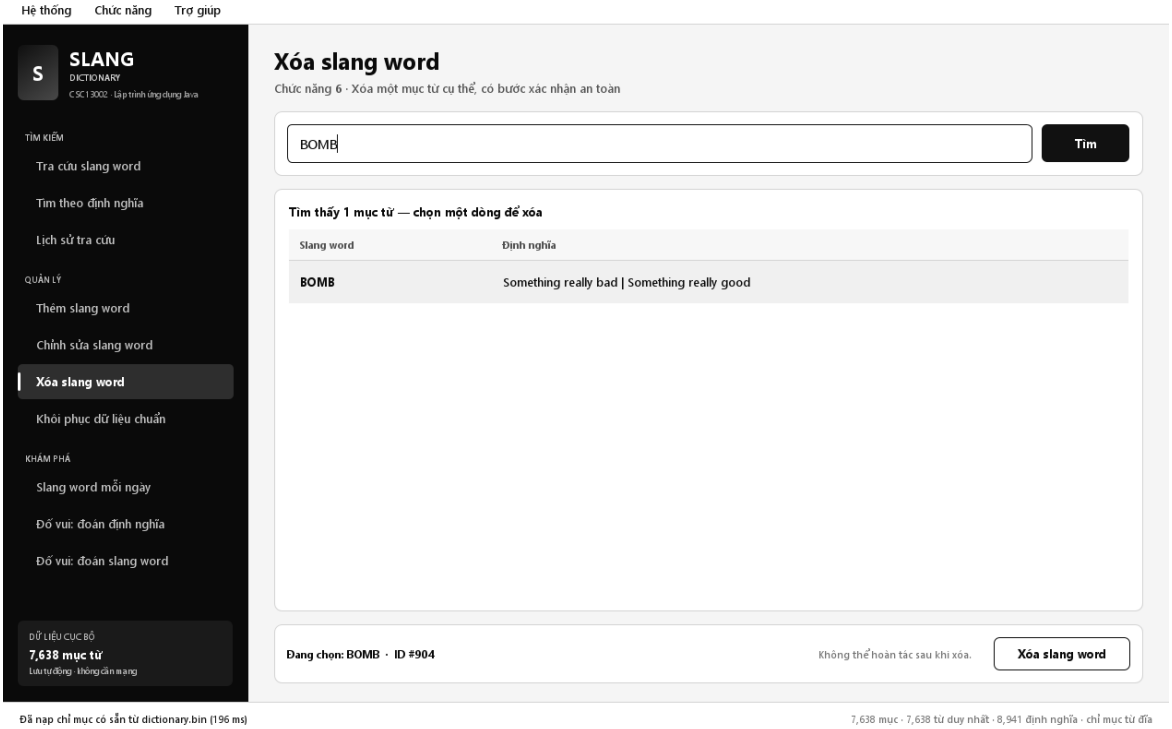
Hình 5.4. Lịch sử tra cứu, mới nhất ở trên cùng



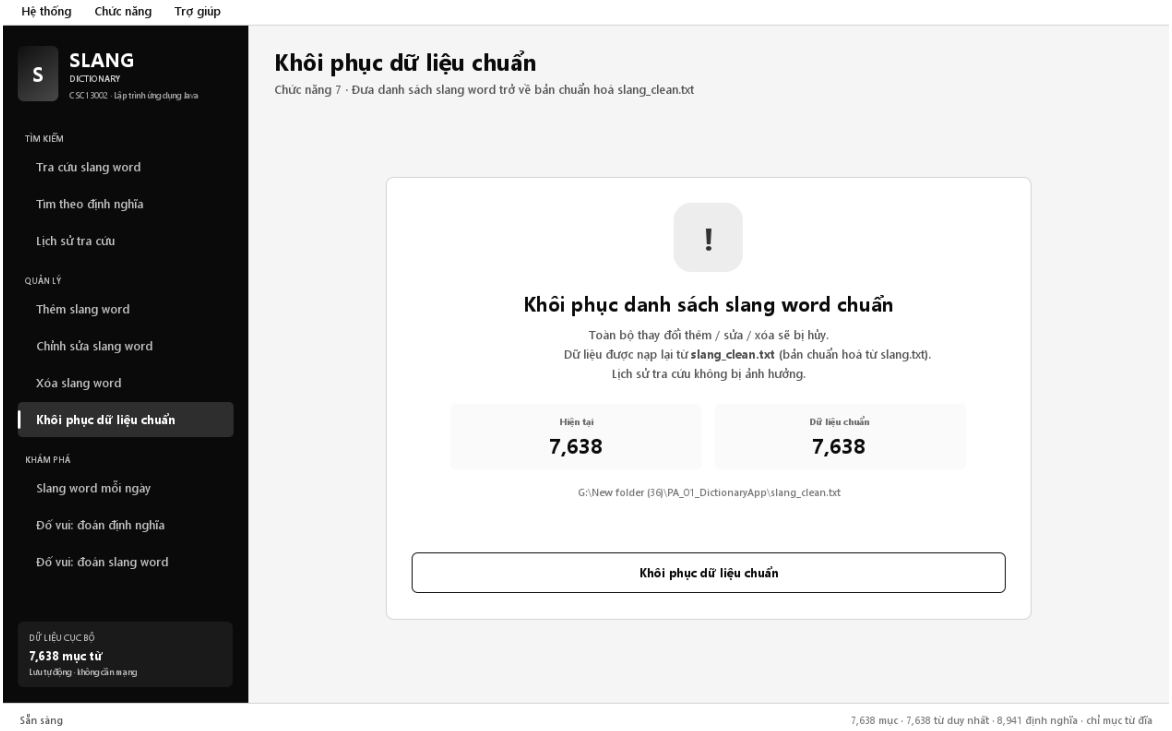
Hình 5.5. Form thêm slang word



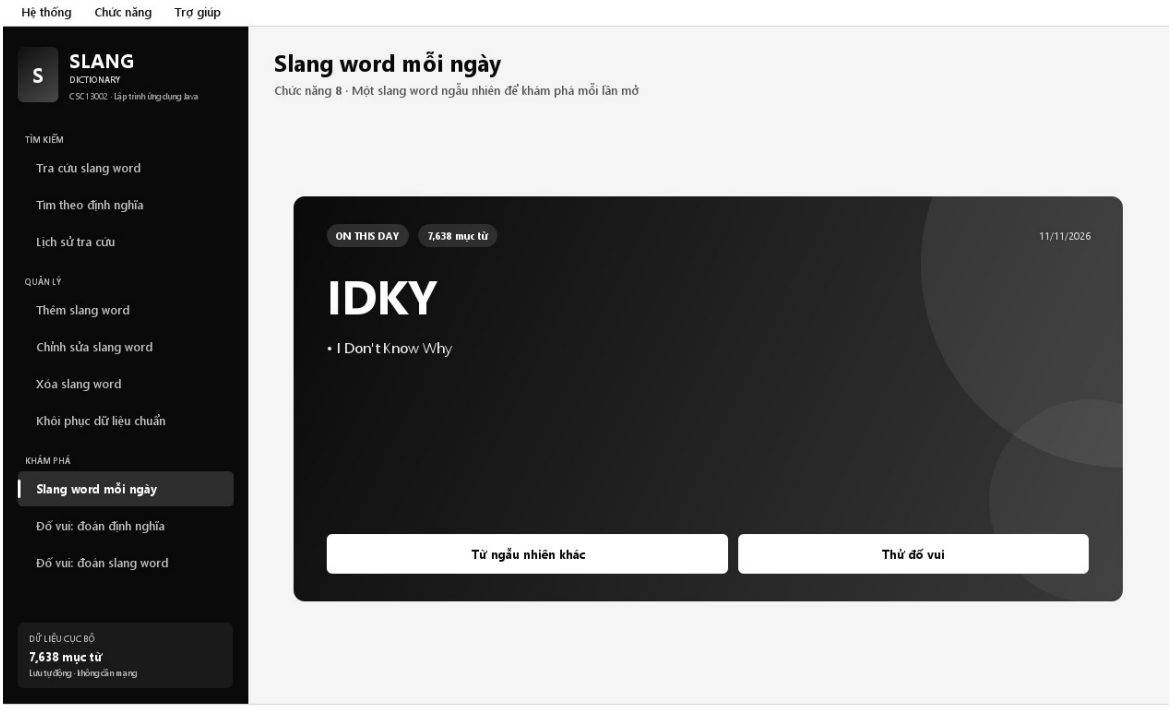
Hình 5.6. Chỉnh sửa mục từ đã chọn



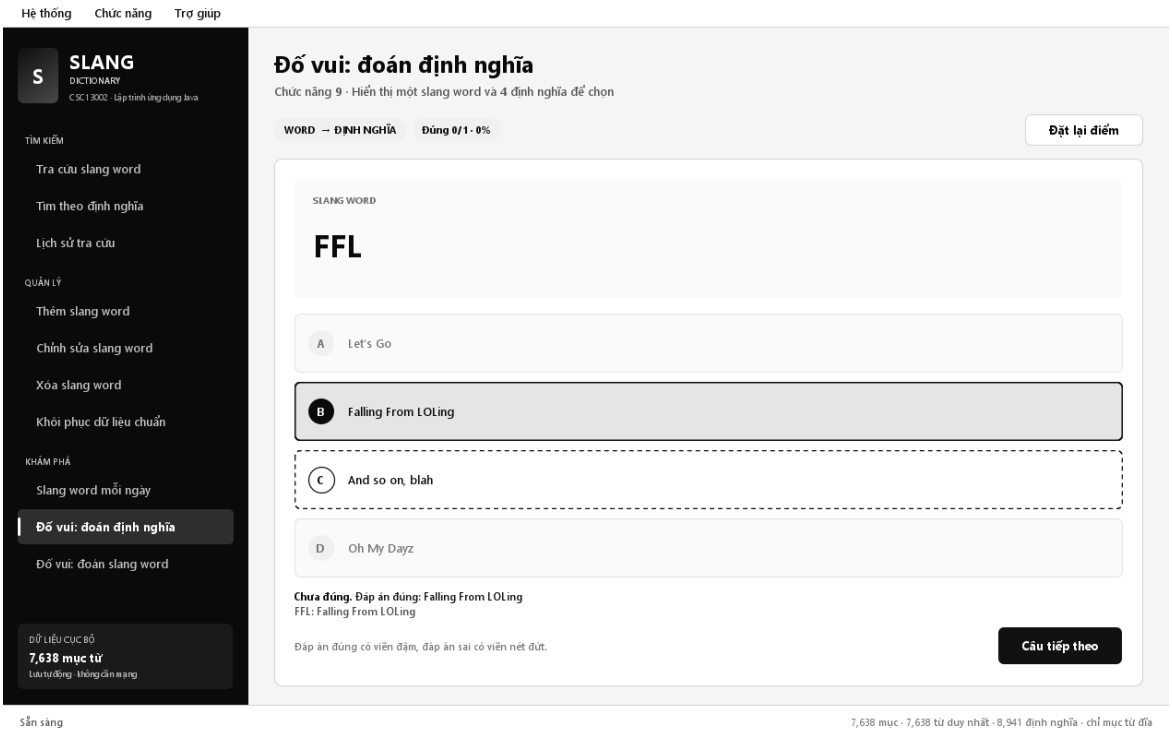
Hình 5.7. Xóa mục từ, có xác nhận trước khi thực hiện



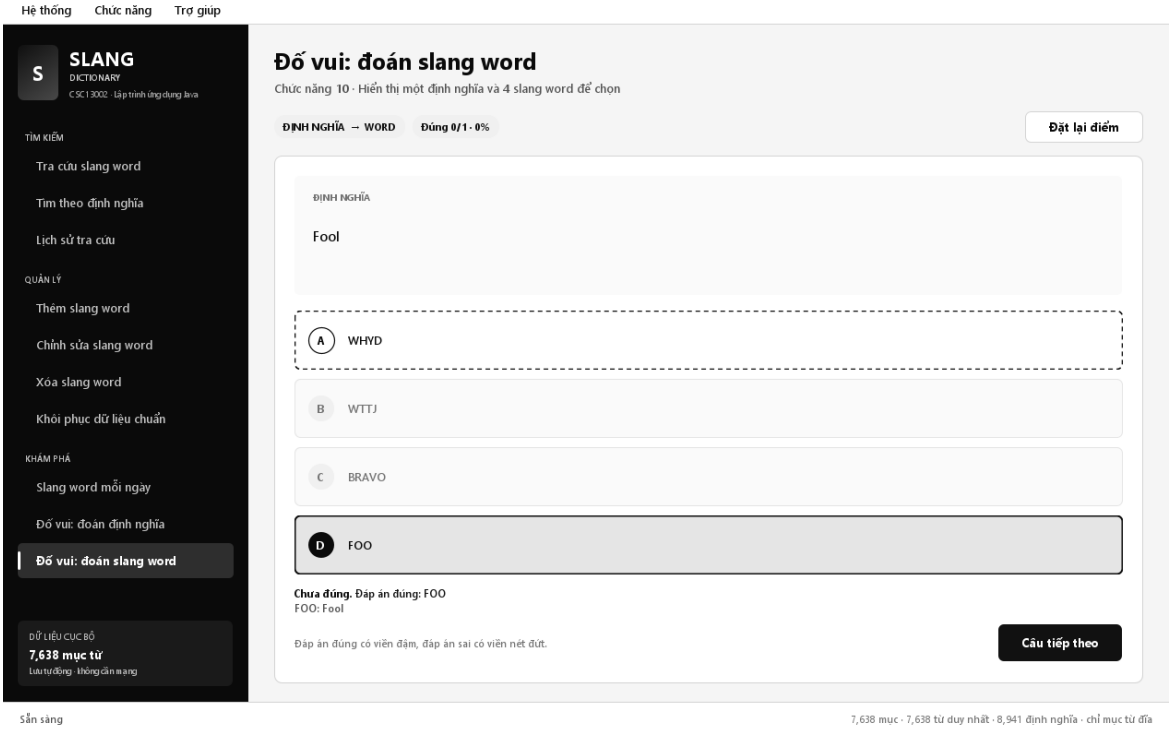
Hình 5.8. Khôi phục danh sách chuẩn từ slang_clean.txt



Hình 5.9. Slang word ngẫu nhiên (on this day)



Hình 5.10. Đố vui word → nghĩa sau khi chọn sai: đáp án đúng viền đậm, đáp án sai viền nét đứt



Hình 5.11. Đố vui nghĩa → word sau khi chọn sai: phản hồi bằng chữ và kiểu viền

5.2. Điều hướng và phím tắt

Phím	Chức năng
Ctrl+1 … Ctrl+0	Mở nhanh 10 chức năng
Ctrl+H	Trang tổng quan
Ctrl+Q	Thoát
Enter	Thực hiện tìm kiếm ở ô nhập

6. KIỂM THỬ VÀ ĐÁNH GIÁ

6.1. Chiến lược kiểm thử

- Kiểm thử đơn vị/tích hợp cho parser, tìm kiếm, thêm/sửa/xóa, lưu trữ, reset, lịch sử, quiz.
- Đối chiếu tìm kiếm definition với brute-force trên dữ liệu thật.
- Đo hiệu năng với 100.000 mục sinh tự động, kiểm tra nạp snapshot không rebuild.
- Smoke test giao diện: mở 11 màn hình, chụp 13 ảnh trạng thái.

6.2. Kết quả kiểm thử tự động

Nhóm	Nội dung	Số kiểm tra
1	Parser, dữ liệu chuẩn, AMA đã gộp	5
2	Tìm theo slang word	7
3	Tìm theo definition + đối chiếu brute-force	20
4	Add / overwrite / duplicate / edit / delete	13
5	Persistence, snapshot, fallback	10
6	Reset dữ liệu chuẩn	7
7	History persistence	10
8	Quiz generation	2
9	Hiệu năng 100.000 slang word	14
	Tổng	88/88 đạt

Kết quả đầy đủ nằm trong report/test-results.txt. Hai cảnh báo fallback snapshot trong quá trình test là có chủ đích; test harness đã nuôi im lặng các cảnh báo này để log sạch.

6.3. Hiệu năng với 100.000 slang word

Phép đo	Kết quả	Yêu cầu
Sinh 100.000 mục	~345 ms	—
Xây dictionary + trigram index	~300 ms	—

Phép đo	Kết quả	Yêu cầu
200 lần tra cứu chính xác	0 ms (tổng)	< 1 giây mỗi lần
Tìm definition “money”	~1 ms (587 kết quả)	< 1 giây
Tìm definition “go”	~20 ms (5.152 kết quả)	< 1 giây
Tìm definition “the”	~7 ms (7.536 kết quả)	< 1 giây
Lưu text + snapshot 14 MB	~400 ms	—
Nạp snapshot lần sau	~600–650 ms, indexLoaded=true	Không rebuild
Tìm kiếm sau khi nạp	0–1 ms	< 1 giây

Bộ nhớ sử dụng khi test 100k khoảng 184 MB. Các phép tìm kiếm đều dưới 1 giây, đáp ứng yêu cầu của đề.

6.4. Kiểm thử giao diện

ScreenshotMode mở cửa sổ thật, đi qua 11 màn hình, gọi dữ liệu mẫu BOMB/money, chụp 13 ảnh (11 màn hình + 2 trạng thái quiz đã trả lời) và thoát. Ảnh được chuyển sang grayscale; kiểm tra bằng Python/Pillow cho thấy 0 pixel lệch kênh màu.

```
SMOKE OK: 11 màn hình, 13 ảnh đã mở/chụp thành công.
```

6.5. Các trường hợp biên

- Từ khoá rỗng, toàn khoảng trắng, hoa/thường, khoảng trắng đầu cuối.
- Từ khoá 1–2 ký tự (fallback quét tuần tự).
- Definition không chứa keyword trả về danh sách rỗng, không lỗi.
- Snapshot sai magic, snapshot cũ, file text thay đổi → rebuild đúng.
- Reset khi có dữ liệu sửa → quay về đúng 7.638 mục chuẩn.
- History vượt 200 mục → cắt mục cũ.
- Quiz với dữ liệu trùng từ → vẫn đủ 4 đáp án duy nhất.

6.6. Hạn chế

- Tìm definition là tìm chuỗi con, chưa phải tìm mờ (fuzzy) hay tìm theo ngữ nghĩa.
- Khi overwrite một từ có nhiều bản trùng, ứng dụng cập nhật mục đầu tiên; các bản trùng khác giữ nguyên.
- Điểm quiz chỉ giữ trong phiên chạy, không lưu qua các lần mở.
- Ảnh chụp commit GitHub và video YouTube là artifact bên ngoài, cần sinh viên hoàn thiện trước khi nộp.

7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án đã xây dựng hoàn chỉnh ứng dụng Slang Dictionary bằng Java Swing với 10 chức năng theo yêu cầu, giao diện đen–trắng nhất quán và kiến trúc tách biệt model/data/service/ui.

Điểm nổi bật là chỉ mục từ khoá HashMap kết hợp chỉ mục nghịch đảo trigram, giúp tra cứu chính xác tức thời và tìm definition trong vài mili giây với 100.000 slang word. Snapshot nhị phân có fingerprint giúp lần chạy sau không phải đánh chỉ mục lại, đồng thời vẫn có cơ chế fallback an toàn khi dữ liệu thay đổi hoặc snapshot hỏng.

Hướng phát triển tiếp theo:

- Bổ sung tìm kiếm mờ, tìm theo nhiều từ khoá và xếp hạng kết quả.
- Cho phép import/export CSV/JSON và đồng bộ dữ liệu.
- Thêm nhiều dạng câu hỏi, độ khó và lưu điểm qua các phiên.
- Đóng gói installer hoặc dùng JavaFX cho trải nghiệm hiện đại hơn.

8. TÀI LIỆU THAM KHẢO

1. Đề bài Đồ án #1 – Slang Word, môn CSC13002 – Lập trình ứng dụng Java (file DAMH#1.pdf).
2. Java SE 17 API Specification: java.util (Collections, HashMap, ArrayDeque), java.io, java.nio.file, javax.swing.
3. Dữ liệu slang.txt do đề bài cung cấp; bản chuẩn hoá là slang_clean.txt.

Tỷ lệ tham khảo mã nguồn bên ngoài: 0%. Ứng dụng chỉ sử dụng JDK chuẩn, không dùng thư viện bên thứ ba và không sao chép mã nguồn từ internet hay từ người khác.

PHỤ LỤC A – TỰ ĐÁNH GIÁ

STT	Chức năng	Điểm	Minh chứng
1	Tìm theo slang word	10/10	Ảnh 5.2; test Search by slang word
2	Tìm theo definition	10/10	Ảnh 5.3; test Search by definition
3	Lịch sử	10/10	Ảnh 5.4; test History persistence
4	Thêm + xử lý trùng	10/10	Ảnh 5.5; test Add/overwrite/duplicate
5	Chỉnh sửa	10/10	Ảnh 5.6; test Edit
6	Xóa + xác nhận	10/10	Ảnh 5.7; test Delete
7	Reset	10/10	Ảnh 5.8; test Reset
8	Random	10/10	Ảnh 5.9

STT	Chức năng	Điểm	Minh chứng
9	Đố vui word → nghĩa	10/10	Ảnh 5.10; 200 câu test
10	Đố vui nghĩa → word	10/10	Ảnh 5.11; 200 câu test
	Tổng	100/100	

PHỤ LỤC B – CHẠY VÀ ĐÓNG GÓI

```
run.bat          # chạy ứng dụng
test.bat         # build + chạy 88 kiểm tra + tạo jar
powershell -NoProfile -ExecutionPolicy Bypass -File package.ps1 ^
  -Mssv <MSSV> -FullName "<Họ tên>" -ClassName "<Lớp>" ^
  -VideoUrl "https://youtu.be/..." -Score 100
```

File nộp có dạng <MSSV>_100.zip, gồm source, jar, dữ liệu, ảnh chụp, kết quả test, file tự đánh giá và kịch bản video.

PHỤ LỤC C – VIDEO VÀ COMMIT

- Link video YouTube: .....
- Ảnh chụp commit: cần chụp trang Commits trên GitHub/Bitbucket/GitLab sau khi push.
- Repo local có lịch sử commit thực tế; ảnh local trong report/commits chỉ là minh chứng tạm.
- Cần đảm bảo commit được phân bố theo các ngày làm việc thực tế trước deadline.